Computers & Security 166 (2026) 104886

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

FFuzz: Function-level execution-path-aware oracle optimization for efficient fuzzing on heterogeneous platforms

Xu Chen [a,b,*], Zhe Pan [a,b], Ruomin Fang [a,b], Liwei Chen [a,b], Gang Shi [a,b]

[a] Institute of Information Engineering, Chinese Academy of Sciences, BeiJing, China
[b] School of Cyber Security, University of Chinese Academy of Sciences, BeiJing, China

ARTICLE INFO

Keywords:
Fuzzing
Test oracle
Execution path
Software testing
Security

ABSTRACT

Greybox fuzzing is a widely used vulnerability detection technique. However, existing methods face challenges in providing an effective oracle for identifying vulnerabilities when testing software on heterogeneous platforms, and enhancing the oracle often introduces unacceptable performance overhead.

To address these challenges, we propose FFuzz, a greybox fuzzing framework that enhances vulnerability detection on heterogeneous platforms while maintaining low overhead. FFuzz analyzes function-level execution paths of test cases on the CPU to identify inputs whose execution behaviors indicate a higher likelihood of triggering vulnerabilities. Based on this analysis, FFuzz selectively applies an enhanced oracle during execution on heterogeneous platforms, thereby improving detection effectiveness without introducing significant performance overhead. In addition, FFuzz incorporates an energy allocation strategy that dynamically assigns fuzzing effort according to each seed's vulnerability exposure potential, further enhancing testing efficiency. We evaluated FFuzz on two widely used software running on heterogeneous platforms, PyTorch and TensorFlow. Compared to Atheris and FreeFuzz, FFuzz improved vulnerability detection capabilities by 227% and 445%, respectively. Compared with the approach that applies the enhanced oracle without distinguishing test cases, FFuzz improved performance by 382%. Additionally, FFuzz discovered 6 new vulnerabilities, all of which were confirmed by developers after reporting.

1. Introduction

Fuzzing is an effective technique for the automated discovery of software vulnerabilities, widely used in both industry (Fioraldi et al., 2020; Zalewski, 2014; Inc, 2016; Serebryany, 2017) and academia (Chen et al., 2025; Lemieux and Koushik, 2018; Zheng et al., 2023; You et al., 2019). Based on the degree of runtime information gathered from the target program, fuzzing is typically classified into blackbox (Woo et al., 2013), whitebox (Ganesh et al., 2009), and greybox (Böhme et al., 2016). Among these techniques, greybox fuzzing strikes a balance between analysis precision and execution efficiency by leveraging lightweight instrumentation to collect runtime coverage information, typically edge coverage (Zalewski, 2014), to guide the testing process. In a coverage-guided greybox fuzzing approach, the fuzzer selects a seed from the seed corpus (a collection of test cases that exhibit new execution behaviors), modifies it according to predefined mutation rules, and generates a large number of test cases to execute the program under test (PUT). Subsequently, based on the runtime feedback of coverage information, the fuzzer determines whether a test case triggers new execution behavior. If a test case exhibits new execution behavior, it is added to the corpus. Through this iterative process, fuzzing gradually explores the potential behavioral space of the PUT, thereby improving the effectiveness of vulnerability discovery.

The effectiveness of fuzzing critically depends on the test oracle, which determines whether a program execution violates expected behavior and thus exposes a vulnerability. Currently, mainstream fuzzing methods typically rely on executing code on the CPU (Central Processing Unit) and detecting crashes as the primary oracle for identifying vulnerabilities (Zalewski, 2014; Fioraldi et al., 2020; Serebryany, 2017). When fuzzing software on heterogeneous platforms, traditional crash detection oracles may be insufficient to fully reveal potential vulnerabilities. As a result, some approaches introduce differential testing as a supplementary oracle (Wei et al., 2022; Li et al., 2025). The basic principle of the test oracle for heterogeneous platforms is illustrated in Fig. 1. First, the generated test cases are executed on the CPU to detect whether the program crashes. Then, the same test cases are executed on the GPU (Graphics Processing Unit), and the results from both the CPU and GPU executions are compared to identify any mismatches, which can then be

* Corresponding author. Correspondence to: Institute of Information Engineering, Shucun Road, Haidian District, Beijing, China.
E-mail address: chenxu@iie.ac.cn (X. Chen).

https://doi.org/10.1016/j.cose.2026.104886
Received 4 December 2025; Received in revised form 2 March 2026; Accepted 3 March 2026
Available online 6 March 2026

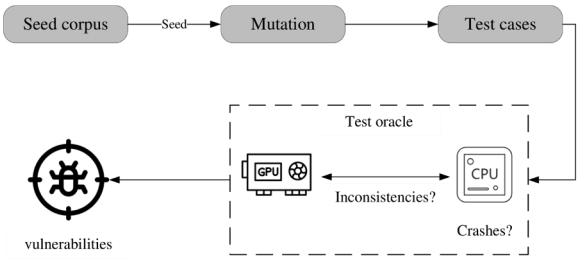


Fig. 1. Overview of the test oracle for heterogeneous platforms.

used to infer potential defects. This combined oracle provides a practical means to detect potential code defects on heterogeneous platforms.

However, existing test oracles exhibit limitations when applied to heterogeneous platforms. Crash detection mainly relies on hardware mechanisms to monitor abnormal program behavior on the CPU (e.g., illegal memory access, division by zero), and the operating system sends signals indicating whether a crash has occurred. Nevertheless, this crash signal-based oracle suffers from false negatives, as it can only detect vulnerabilities that trigger explicit crashes (Kong et al., 2025). For programs running on non-CPU platforms such as GPUs, the lack of corresponding hardware mechanisms and exception feedback signals makes it difficult for fuzzers to capture potential crash events (Guo et al., 2024). Furthermore, while differential testing can partially mitigate these limitations, it is prone to false positives due to the accumulated floating-point precision loss (Wei et al., 2022). Although sanitizers (Serebryany, 2012; Lee et al., 2015; Corporation, 2023) can enhance runtime vulnerability detection and reduce false negatives, their use introduces significant performance overhead, which further impacts the efficiency and scalability of fuzzing.

To address these challenges, this paper introduces FFuzz, a novel greybox fuzzing framework designed to enhance vulnerability detection on heterogeneous platforms while controlling performance overhead. FFuzz primarily focuses on traditional crashes and memory-related vulnerabilities detected by runtime sanitizers, such as use-after-free, heap or stack buffer overflows, and illegal or uninitialized memory accesses on GPU devices. To this end, FFuzz integrates runtime sanitizers to reduce the false negatives of traditional CPU-based oracles and to compensate for the lack of exception feedback mechanisms on GPUs. However, enabling sanitizers for all executions would incur unacceptable performance overhead. To mitigate this, FFuzz differentiates test cases based on their function-level execution paths and applies the high-overhead enhanced oracle only to those with a higher likelihood of triggering vulnerabilities, achieving a balance between detection capability and performance cost. In addition, FFuzz allocates energy according to a seed's potential impact on exposing vulnerabilities, improving the efficiency of resource utilization and further enhancing overall fuzzing performance.

We implemented a prototype of FFuzz and evaluated it on PyTorch and TensorFlow, two widely used software frameworks for heterogeneous platforms. Results show that FFuzz not only significantly enhances vulnerability detection capability but also introduces only modest performance overhead. Compared with state-of-the-art fuzzers, Atheris (Google, 2020) and FreeFuzz (Wei et al., 2022), FFuzz improves vulnerability detection capability by 227% and 445%, respectively. Moreover, in mitigating the performance overhead of enhanced oracles, FFuzz achieves a performance improvement of 382%. Its energy allocation strategy further enhances fuzzing efficiency, contributing an additional 10.1% improvement in edge coverage and 16.5% in vulnerability detection.

In summary, this paper makes the following contributions:

- We integrate runtime sanitizers into heterogeneous platform software to enhance vulnerability detection, effectively mitigating false negatives on CPUs and addressing the lack of exception feedback on GPUs.
- We propose a mechanism that differentiates test cases based on their function-level execution paths, enabling selective use of high-overhead enhanced oracles for inputs with a higher likelihood of triggering vulnerabilities.
- We introduce an energy allocation strategy that improves resource utilization and accelerates the fuzzing process.
- We implement a prototype of FFuzz and conduct extensive evaluations, demonstrating 227% improvement in vulnerability detection, 382% performance gain, and the discovery of six previously unknown vulnerabilities. We have released our artifact at https://github.com/cx104906/FFuzz.

2. Background

2.1. Heterogeneous platform software

Heterogeneous platform software refers to programs that utilize multiple types of computing resources, such as CPUs and GPUs, to accelerate computation. Representative examples include popular frameworks such as PyTorch (Team, 2017) and TensorFlow (Team, 2015), numerical libraries like NumPy (Harris et al., 2020) with GPU support, and OpenCL-based scientific computing applications (Stone et al., 2010). These programs typically provide functionally equivalent implementations for both CPU and GPU execution, allowing users to exploit GPU parallelism for performance while maintaining CPU execution for compatibility, debugging, or portability. Since CPU and GPU implementations are expected to perform the same functionality for the same inputs, inconsistencies between them may indicate potential defects. Therefore, existing approaches employ differential testing between CPU and GPU outputs as a supplementary oracle to identify heterogeneous platform inconsistencies (Deng et al., 2023).

From the perspective of fuzzing, CPU execution can be instrumented to collect runtime coverage information. In contrast, GPU execution generally lacks lightweight instrumentation mechanisms, making it difficult to obtain coverage information directly. As a result, traditional coverage-guided fuzzing methods cannot fully observe GPU-side behavior. Nevertheless, although CPU and GPU implementations may differ internally, executing the same input on the CPU often provides useful indications of how the corresponding GPU execution may behave. This motivates the design of FFuzz. In the absence of direct observability of GPU-side execution, FFuzz uses CPU-side execution behavior to approximate GPU behavior. Specifically, FFuzz leverages execution information obtained from CPU runs as an indicator of a test case's likelihood of triggering vulnerabilities on the GPU. Such information serves as the basis for selectively applying enhanced oracles, thereby enabling more effective and efficient vulnerability detection on heterogeneous platforms.

2.2. Sanitizers

Sanitizers are runtime tools designed to detect vulnerabilities and undefined behaviors in software execution, enabling more comprehensive vulnerability detection than traditional crash-based methods (Serebryany, 2012; Lee et al., 2015). On CPUs, AddressSanitizer (ASan) inserts memory-access checks at compile time to detect memory safety issues such as buffer overflows, use-after-free, and invalid memory accesses (Serebryany, 2012). UndefinedBehaviorSanitizer (UBSan) identifies various forms of undefined behavior in C/C++ programs (Lee et al., 2015), including integer overflows, invalid type casts, and misaligned pointer operations. For GPU programs, NVIDIA Compute Sanitizer (CSan) (Corporation, 2023) provides runtime detection of out-of-bounds accesses, misaligned memory operations, and data races in CUDA kernels, offering diagnostic feedback otherwise unavailable on GPUs.

```
import torch
m1 = torch.randn(2, 291105, 1).to_sparse().cuda()
m2 = torch.randn(2, 1, 1).cuda()
torch.bmm(m1, m2) # triggers a GPU-specific memory vulnerability
```

Fig. 2. An example of a GPU-only memory safety violation in heterogeneous platform software.

These sanitizers significantly improve vulnerability detection and enhance the effectiveness of fuzzing on both CPU and GPU programs.

3. Motivation

3.1. GPU-only vulnerabilities

Heterogeneous platform software may exhibit vulnerabilities that are only observable when executing on GPUs. In particular, the same test case may execute correctly on the CPU, even with sanitizer instrumentation enabled, while triggering memory safety violations on the GPU due to differences in kernel implementations, memory layouts, and execution models.

Fig. 2 shows a simplified example adapted from a real-world PyTorch issue. The test case performs batched matrix multiplication on sparse tensors. When executed on the CPU, the program completes normally and no memory errors are reported, even with AddressSanitizer enabled. In contrast, executing the same test case on the GPU triggers an invalid global memory write vulnerability, which can only be detected by Compute Sanitizer.

This example illustrates that CPU-only fuzzing is insufficient for uncovering certain GPU-specific failures. Consequently, effective vulnerability detection for heterogeneous platform software requires fuzzing techniques that explicitly exercise and observe GPU executions.

3.2. Sanitizer overhead

Although sanitizers improve vulnerability detection, enabling them directly during fuzzing introduces substantial performance overhead. ASan relies on compile-time instrumentation, inserting a large amount of checking logic into the PUT, which increases instruction count and causes significant slowdown. On the GPU side, CSan performs vulnerability detection through runtime memory-access monitoring rather than compiler instrumentation; however, this mechanism still incurs considerable runtime slowdown, increased memory usage, and large volumes of diagnostic output under certain detection modes. Furthermore, CSan operates as an external tool, meaning that each detection task introduces additional tool-startup overhead.

To quantify these costs, we evaluated the performance impact of using ASan and CSan on heterogeneous platform software such as PyTorch and TensorFlow. The evaluation was conducted using the corpus of inputs retained from baseline Atheris fuzzing runs. These inputs were replayed on the program under test in the Default configuration (without sanitizers), as well as in configurations with ASan and CSan enabled, and the average execution speed was measured under an identical input set. Specifically, the reported execution speeds correspond to the average number of test cases executed per second. As shown in Table 1, ASan introduces an average overhead of 379%, while CSan produces an even higher runtime overhead of 8511%. Due to these significant and often unacceptable performance penalties, existing fuzzing frameworks typically avoid enabling sanitizers by default. As a result, existing frameworks face a trade-off between fuzzing efficiency and vulnerability detection. Addressing this trade-off is one of the motivations behind the design of FFuzz.

Table 1
Performance overhead of AddressSanitizer and compute sanitizer on heterogeneous platform software.

Target	Default	AddressSanitizer		Compute Sanitizer	
	Speed	Speed	Slowdown	Speed	Slowdown
PyTorch	30.5	8.5	358%	0.35	8714%
TensorFlow	46.2	11.7	394%	0.55	8400%
Average	38.3	10.1	379%	0.45	8511%

```
1  void unique_consecutive(/*...*/){
2      /*...*/
3      for (const auto i : c10::irange( numel)) {
4          if (input_data[i] != *p) {
5              *(++p) = input_data[i];
6              if (return_counts) {
7                  *(q++) = i - last;
8                  last = i;
9              }
10             if (return_inverse) {
11                 inverse_data[i] = input_data[i
                       + q];
12             }
13         }
14     }
15     /*...*/
16 }
```

Listing 1
A simplified code snippet extracted from `PyTorch`.

3.3. Vulnerability indicators

Fuzzing generates a large number of test cases for the PUT to execute. Among these inputs, only a small portion may trigger vulnerabilities, and they often exhibit unusual execution behaviors (Jiang et al., 2021; Kong et al., 2025). One such indicator is the presence of a unique execution path (Kong et al., 2025), which frequently correlates with the likelihood of exposing a hidden vulnerability.

Listing 3.3 shows a simplified vulnerability example extracted from PyTorch, adapted from a real-world bug report (GitHub issue #71089). The function iterates over consecutive unique elements and optionally computes additional metadata depending on two flags: `return_counts` and `return_inverse`. When `return_counts` is true, the variable `q` is incremented to track positions of unique elements at line 7. However, if `return_inverse` is also true, the assignment at line 11 may access memory beyond the bounds of `input_data`. Setting either flag individually does not trigger the out-of-bounds access; only when both are true does execution follow this specific branch, resulting in a unique path that exposes the hidden vulnerability.

This demonstrates that unique execution paths can serve as effective indicators of potential vulnerabilities. In the context of FFuzz, tracking such paths provides valuable guidance for distinguishing test cases that are more likely to reveal subtle vulnerabilities.

4. Methodology

4.1. Overview

Fig. 3 illustrates the overall framework of FFuzz. Similar to traditional fuzzing frameworks, FFuzz selects seeds from the corpus and applies diverse mutation strategies to generate a large number of test cases, which are executed on the CPU side of the PUT. After execution, FFuzz checks whether any new edges are covered. Test cases that exhibit new coverage behaviors are added back into the corpus as valuable seeds to further expand the exploration space. Meanwhile, if a crash is triggered

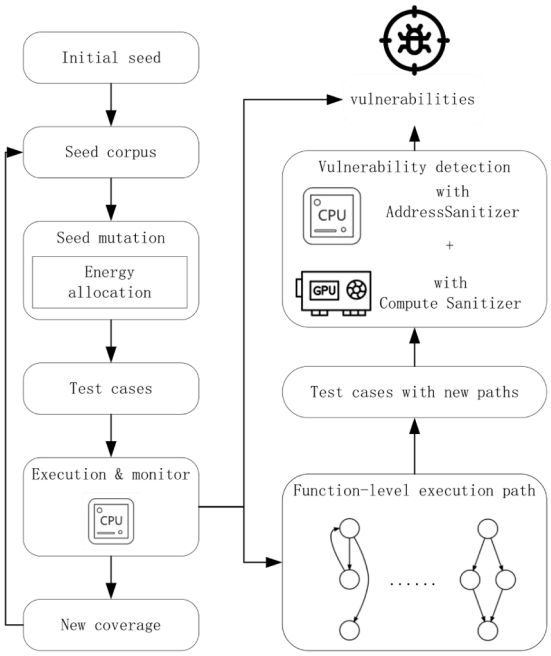


Fig. 3. Architecture of FFuzz.

during CPU-side execution, FFuzz records the corresponding test case for subsequent vulnerability analysis.

To enhance overall vulnerability detection capability, FFuzz integrates sanitizer mechanisms to mitigate potential false negatives on the CPU side and to capture vulnerabilities that may arise on the GPU side. However, enabling high-overhead sanitization checks throughout the entire fuzzing process would introduce unacceptable performance overhead. Therefore, after each test execution, FFuzz distinguishes test cases based on their function-level execution paths. If a test case exhibits a previously unseen function-level path, FFuzz considers it more likely to trigger vulnerabilities and applies high-overhead detection only to these high-potential samples. Through this tiered detection strategy, FFuzz achieves an effective balance between detection capability and overall execution efficiency. In addition, FFuzz adopts an energy allocation strategy based on the potential of each seed to trigger vulnerabilities, assigning more fuzzing resources to higher-potential seeds and further improving the efficiency of fuzzing and the ability to uncover vulnerabilities.

In the following sections, we detail the design components of FFuzz, including the enhanced oracle mechanism for improved vulnerability detection, the function-level execution path-based test case differentiation method, and the energy allocation strategy targeting high-potential seeds.

4.2. Test oracle

The test oracle in FFuzz is designed to determine whether a generated test case triggers potential vulnerabilities during execution. Similar to traditional approaches, FFuzz first executes all test cases on the CPU and uses crash detection to identify those that expose explicit vulnerabilities for further analysis. For test cases that may trigger vulnerabilities without causing observable crashes, FFuzz employs an enhanced test oracle.

To reduce false negatives on the CPU side, FFuzz integrates ASan, one of the most widely used sanitizers for detecting memory errors such as stack overflows and heap buffer overflows. Given that certain vulnerabilities are only triggered on the GPU side, FFuzz leverages CSan to identify such GPU-specific issues. As CSan is an external tool, FFuzz converts relevant test cases into GPU-executable data structures and invokes CSan for memory error checking. FFuzz then analyzes CSan's output to determine whether a GPU-side vulnerability has been triggered.

It is worth noting that differential testing between the CPU and GPU often produces false positives. The main reasons for these false positives include differences in execution behavior between the CPU and GPU, accumulated floating-point precision loss, and other factors. Because inconsistent results do not reliably indicate the presence of a vulnerability, FFuzz does not perform differential comparisons between CPU and GPU execution results. Instead, it adopts an enhanced oracle mechanism to determine whether a vulnerability exists. FFuzz integrates ASan and CSan to detect potential memory errors and illegal behaviors on the CPU and GPU sides, respectively. With the help of these two types of sanitizers, FFuzz can directly determine whether a test case has triggered a vulnerability on each platform. This approach enables FFuzz to make vulnerability triggering judgments more directly and effectively, largely replacing the role of differential testing and significantly reducing its false positives.

4.3. Function-level execution path

Execution paths provide an effective indicator of whether a test case may trigger a potential vulnerability. However, capturing the full execution path inevitably leads to the path explosion problem (Wang et al., 2019; Kong et al., 2025). To address this, FFuzz implements a function-level execution path to distinguish test cases and identify those worthy of further verification for potential vulnerabilities. This design strikes a balance between path tracking granularity and performance overhead.

4.3.1. Instrumentation

The function-level execution path mechanism in FFuzz is designed to capture the execution behavior within each function at an appropriate level of granularity, enabling more accurate differentiation of test cases. To achieve this, FFuzz extends the traditional shared memory used by the PUT. In addition to recording basic edge coverage information, the system allocates an extra shared memory region dedicated to maintaining function-level execution path hashes. As illustrated in Fig. 4, FFuzz adopts a function-level hashing partition strategy to manage this extended region: each function is assigned an independent shared memory segment to record its internal execution path. At runtime, execution path information from different functions is written into their respective segments, forming a structured and function-isolated path recording mechanism. This design not only avoids the path explosion associated with global execution path tracking but also enhances the discriminability of path information, providing a more reliable basis for distinguishing test cases.

To support this mechanism, FFuzz applies lightweight instrumentation to the PUT. The instrumentation logic, shown in Fig. 5, works as follows: when a basic block is executed, the system first loads the current path hash of the corresponding function (`prev_hash`) from its dedicated shared memory segment. The instrumentation then XORs (Exclusive OR) `prev_hash` with a randomly generated value produced during compilation, yielding the function-level path hash for the current basic block (`cur_hash`). This `cur_hash` is written back to the same shared memory location and serves as the basis for subsequent updates by later basic blocks within the same function. Through this rolling-style hash updating process, FFuzz efficiently constructs function-level execution paths, which are later used for test case filtering and potential vulnerability triggering analysis.

Since the number of execution path hashes can grow exponentially with the size of a function, FFuzz allocates multiple shared memory segments for functions exceeding a certain threshold. This ensures that the basic blocks of larger functions are distributed across segments, preventing excessive state explosion.

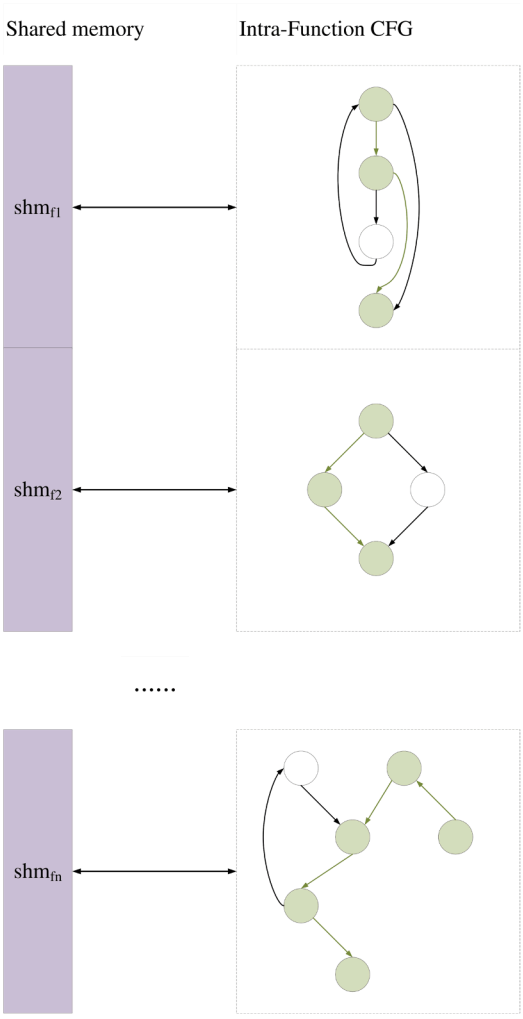


Fig. 4. Partitioned shared memory for recording intra-function execution paths.

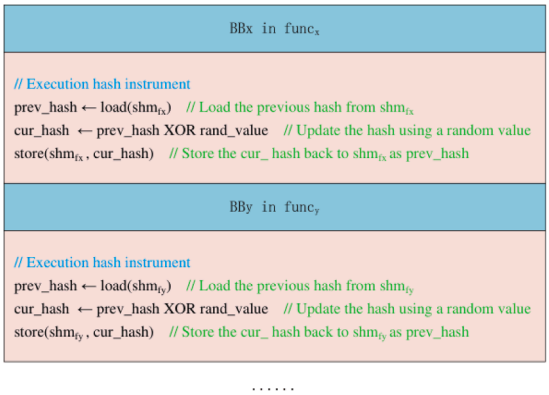


Fig. 5. Pseudocode of FFuzz's function-level instrumentation.

4.3.2. Detection of new paths

Each function has its own dedicated shared memory segment to record the execution paths within that function. During execution, the path information of a function is continuously updated and stored in the corresponding shared memory region. Based on this design, FFuzz checks the shared memory segments after each test execution to efficiently detect new execution paths. The purpose of detecting new paths is to identify test cases that trigger previously unrecorded paths, thereby providing clues for potential vulnerability discovery.

Algorithm 1 New path detection.

Input: Function shared memory segments SM
Input: Path dictionary $Dict$
Output: New path records

```
for segment ∈ SM do
  for (index, value) in segment do
    if value ≠ 0 then
      if index ∉ Dict then
        Add index ↦ {value} to Dict
        Mark this path as new
      else
        if value ∉ Dict[index] then
          Add value to Dict[index]
          Mark this path as new
        end if
      end if
    end if
  end for
end for
```

The path detection mechanism in FFuzz works as follows. After each execution, FFuzz iterates over all shared memory segments and checks the path information stored in each segment. If the value in a segment is non-zero, it indicates that the corresponding function was involved in a path during this execution. FFuzz then looks up the path index in the maintained dictionary. Specifically, in this dictionary, the keys are path indexes of functions, and the values are sets that record the hash values of the paths corresponding to those indexes. If the path index is not found in the dictionary, it means this is a new path, and FFuzz adds the index and hash value to the dictionary. If the path index exists, FFuzz checks whether the current path hash value is already in the set corresponding to that index. If the hash value is not found, it indicates a new path, and FFuzz adds the hash value to the set and marks the path as new. Consequently, whenever a new path is detected, FFuzz marks the corresponding test case, providing a foundation for subsequent vulnerability analysis. Overall, this mechanism improves the accuracy of vulnerability detection and aids in optimizing the fuzzing process.

To formalize the detection process described above, Algorithm 1 outlines FFuzz's path detection mechanism. Specifically, the algorithm traverses all shared memory segments and examines the recorded path values. If a segment contains a non-zero value, it indicates that a function-level execution path has been exercised during the current test case. By checking the path index against the dictionary, FFuzz determines whether the path is new or has already been recorded. If it is a new path, FFuzz adds the corresponding path hash value to the dictionary and marks it for further analysis.

4.4. Energy allocation

In the fuzzing process, efficient energy allocation is critical to the effectiveness of the testing strategy. Here, energy refers to the computational resources allocated to each seed, which influences the seed's subsequent mutations and exploration of execution paths. The goal is to allocate more energy to seeds that are more likely to discover new paths and trigger potential vulnerabilities, while ensuring that resources are distributed efficiently across the entire corpus.

The energy allocation for each seed s is determined by the following formula:

$$E(s) = \text{clamp}\big(\lfloor E_0(1+\gamma S(s))\rfloor,\ E_{\min},\ E_{\max}\big). \tag{1}$$

Specifically, $E(s)$ is the final energy allocated to seed s, E_0 is the base energy, γ is a hyperparameter, and $S(s)$ quantifies the potential of seed s in generating new execution paths and interesting mutations. The result is then constrained within the limits defined by $E_{\min}$ and $E_{\max}$,

which represent the minimum and maximum allowable energy values, respectively.

The score $S(s)$ for seed s is calculated as a weighted sum of three factors:

$$S(s) = w_1\, D(f) + w_2\, \tilde{I} + w_3\, \tilde{N}, \tag{2}$$

where $D(f)$ is a function that represents the effectiveness of the seed's mutation, $\tilde{I}$ is the normalized number of new interesting seeds generated from the mutations of seed s, $\tilde{N}$ is the normalized number of new execution paths discovered by seed s, w_1, w_2 and w_3 are weight hyperparameters that balance the contribution of each factor.

In FFuzz, an interesting seed is defined in the AFL sense: a seed whose execution covers previously unseen edges. Although interesting seeds overlap with new execution paths, as new edges imply new paths, the two are not equivalent. New execution paths also include distinct internal control-flow behaviors without introducing new edges. By assigning additional energy to interesting seeds, FFuzz allocates a larger mutation budget to seeds that contribute to expanding code coverage, while still valuing seeds that contribute to exposing new execution paths without new edges.

The function $D(f)$ quantifies a seed's mutation effectiveness based on the number of times it has been fuzzed. It is given by:

$$D(f) = \frac{1}{1 + \beta f}, \tag{3}$$

where f is the number of times the seed has been fuzzed (mutated), and β is a constant that determines how quickly $D(f)$ decays as the seed undergoes repeated fuzzing. A lower number of mutations results in a higher $D(f)$, indicating that the seed's mutation is more likely to be effective and should be allocated more energy.

The counts of new interesting seeds I and new execution paths N are normalized to ensure that they fall within a common range. The normalization is performed as follows:

$$\tilde{I} = \frac{I'}{I'_{\max} + \delta}, \qquad \tilde{N} = \frac{N'}{N'_{\max} + \delta}, \tag{4}$$

where

$$I' = \log(1 + I), \qquad N' = \log(1 + N), \tag{5}$$

are the logarithmic transformations of the raw counts of interesting seeds and new paths, respectively. These transformations help mitigate the effect of large numbers and scale the values appropriately. The maximum values, $I'_{\max}$ and $N'_{\max}$ represent the largest I' and N' across all seeds in the corpus, ensuring that the normalization is relative to the most successful seeds.

The maximum values are computed as:

$$\begin{aligned} I'_{\max} &= \max_{s \in \text{corpus}} \log(1 + I_s), \\ N'_{\max} &= \max_{s \in \text{corpus}} \log(1 + N_s). \end{aligned} \tag{6}$$

In summary, the energy allocation mechanism in fuzz testing allows more resources to be directed towards seeds that are likely to produce valuable results, such as discovering new execution paths or generating interesting mutations. By balancing the seed performance score with factors like mutation effectiveness, new interesting seed generation, and path discovery, FFuzz optimizes its exploration of the input space and ensures more efficient vulnerability detection.

5. Implementation and evaluation

FFuzz's implementation consists of two main components. One is the compile-time instrumentation of the target program. The instrumentation is done using LLVM passes, requiring around 500 lines of C/C++ code. The other component is the fuzzing engine. FFuzz incorporates a coverage-feedback fuzzing engine designed for testing heterogeneous platform software. To support its functionality, FFuzz implements approximately 1500 lines of Python code, introducing additional features tailored to its specific goals. In our implementation, FFuzz employs ASan for CPU-side execution and CSan for GPU-side execution to enhance the detection of memory-related errors. Both sanitizers are mature and widely supported on mainstream CPU architectures and NVIDIA GPU platforms, respectively. We evaluated FFuzz through experiments to answer the following questions:

- **RQ1.** How does FFuzz perform in terms of edge coverage?
- **RQ2.** Does FFuzz improve vulnerability discovery capabilities?
- **RQ3.** How effective are the function-level execution paths?
- **RQ4.** Does the energy allocation strategy lead to improvements?
- **RQ5.** What is the system overhead of FFuzz?

5.1. Evaluation setups

5.1.1. Evaluation benchmarks

We selected PyTorch (v2.8.0) and TensorFlow (v2.19.0) as the test subjects for evaluation. These are two widely used and popular software libraries, with over 95k and 193k stars on GitHub, respectively. Furthermore, PyTorch and TensorFlow are large-scale frameworks, containing 1593 and 3316 APIs, respectively. Testing all APIs would be impractical due to the sheer number of APIs involved. Therefore, to ensure the validity of the evaluation, we selected 50 APIs from each framework that are more prone to vulnerabilities, aiming to focus on a representative and security-relevant subset for assessment. Specifically, we first prioritized APIs that are associated with known vulnerability reports that have not yet been fixed by developers. In addition, we manually analyzed APIs that are structurally or semantically similar to these vulnerable APIs. The identification of similar APIs was conducted through manual inspection of the API documentation and source code. These APIs often share similar implementation patterns and therefore may exhibit potential vulnerability risks. For example, given that operators such as `Conv2d` have been reported to contain vulnerabilities, we also selected related APIs including `Conv1d` and `Conv3d`. The list of selected APIs is provided in the artifact.

For each selected API, FFuzz launches an independent fuzzer instance that targets only that API. Each fuzzer instance invokes a single target API, while auxiliary APIs (e.g., for tensor creation, list construction) are called to assemble valid inputs from mutated data. These auxiliary API calls are not manually written for individual target APIs; instead, FFuzz probabilistically combines generic auxiliary construction methods and retains the combinations that successfully invoke the target API for reuse. These auxiliary calls are considered part of input preparation and are excluded from feedback collection. FFuzz limits coverage and execution-path feedback exclusively to the execution of the target API, ensuring that the fuzzing process focuses on the target API's internal behavior.

5.1.2. Evaluation baselines

We compare FFuzz with two representative fuzzers, Atheris (Google, 2020) and FreeFuzz (Wei et al., 2022). Atheris is an advanced framework for coverage-guided fuzz testing of software that provides Python APIs. FreeFuzz, on the other hand, has been extensively evaluated on PyTorch and TensorFlow and is considered a state-of-the-art fuzzer. In addition to these external baselines, we include several FFuzz variants to further analyze the effectiveness of different design choices. We implement FFuzz-c, a variant that approximates execution paths using edge execution counts (Kong et al., 2025). We also include FFuzz-n, which distinguishes test cases solely based on newly triggered edges. Furthermore, we evaluate FFuzz-a, a high-overhead variant that enables sanitizer checks for all test cases.

Table 2
Comparison of the edge coverage between FFuzz and baselines.

Target	Atheris	FreeFuzz	FFuzz-a	FFuzz-c	FFuzz-n	FFuzz
PyTorch	11,759	7407	3703	8231	11,171	10,583
TensorFlow	20,498	13,373	4429	14,758	19,063	18,038
Avg.	16,128	10,390	4066	11,494	15,117	14,310

5.1.3. Experiment environment

All fuzzing experiments were initialized with an empty seed. All evaluations were conducted on an Intel Xeon Silver 4216 server with 32 CPU cores, 64 GB of RAM, and an NVIDIA Tesla T4 GPU, running Ubuntu 18.04 LTS. Each fuzzer was executed in a Docker container with single-core CPU isolation. The experiment was repeated 10 times, with each run lasting 1 h, and the results were averaged across these repetitions.

5.2. Edge coverage evaluation

Code coverage is an important metric for evaluating a fuzzing framework, as vulnerabilities can only be exposed when the corresponding code regions are executed. Therefore, we evaluate the edge coverage, which is the most widely used form of code coverage, achieved by FFuzz and several baselines, including the fuzzers Atheris and FreeFuzz, as well as three variants: FFuzz-c (which differentiates test cases using edge execution count approximation), FFuzz-n (which differentiates test cases based on whether they trigger new edges), and FFuzz-a (which enables sanitizer checks for all test cases). We report the average edge coverage achieved on both PyTorch and TensorFlow.

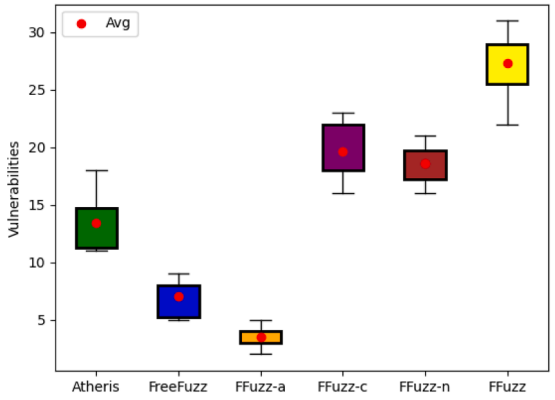
As shown in Table 2, Atheris achieves the highest edge coverage among all evaluated fuzzers, with an average of 16,128 executed edges. Due to methodological differences, FreeFuzz attains only 10,390 edges. In contrast, all FFuzz variants exhibit varying degrees of coverage reduction, as the enhanced oracle checks incorporated during fuzzing reduce execution throughput. In particular, FFuzz-a applies high-overhead oracle checks to all test cases and therefore achieves only 4066 edges on average under limited fuzzing resources, corresponding to only 25% of Atheris's coverage. By comparison, distinguishing test cases and enabling high-overhead oracles only for selected inputs alleviate efficiency degradation.

The default FFuzz implementation, which differentiates test cases based on function-level execution paths, achieves an average coverage of 14,310, approximately 89% of Atheris. FFuzz-c and FFuzz-n reach 71% and 93% of Atheris's coverage, respectively. If the function-level execution path guided oracle ultimately proves to enhance vulnerability discovery, then this moderate reduction, while still retaining 89% of Atheris's coverage, would be considered acceptable.

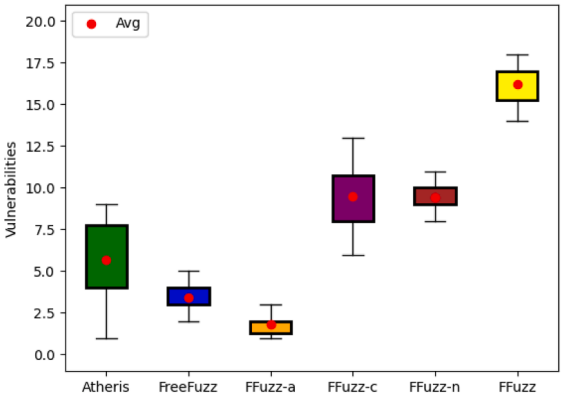
5.3. Vulnerability discovery capability

The primary goal of a fuzzing framework is to uncover potential vulnerabilities in the target program. Consequently, an important metric for evaluation is its vulnerability discovery capability. To assess this, we compare FFuzz against several baselines, including the fuzzers Atheris and FreeFuzz, as well as the variants FFuzz-c, FFuzz-n, and FFuzz-a. This comparison enables us to evaluate the effectiveness of our proposed function-level execution path guided oracle optimization.

To ensure a fair comparison, discovered vulnerabilities are deduplicated before analysis. Specifically, test cases are first grouped according to the vulnerability types reported by the sanitizers, and are then further deduplicated based on the triggering locations and the corresponding call stacks, in combination with manual analysis. As shown in Fig. 6, the results are based on the average number of unique vulnerabilities discovered across the 50 selected APIs from both PyTorch and TensorFlow, after deduplicating the test cases that triggered vulnerabilities. The results show that the default FFuzz configuration discovers the most



(a) PyTorch vulnerability discovery.



(b) TensorFlow vulnerability discovery.

Fig. 6. Comparison of the discovery of vulnerabilities between FFuzz and baselines.

vulnerabilities among all evaluated fuzzers, identifying 27.3 vulnerabilities on average in PyTorch and 16.2 in TensorFlow. Compared with the second-best variant, FFuzz-c, which finds 19.6 vulnerabilities in PyTorch and 9.5 in TensorFlow, FFuzz achieves an average improvement of 55%. When compared with Atheris, which discovers 13.4 vulnerabilities in PyTorch and 5.7 in TensorFlow, FFuzz achieves an improvement of 227% on average.

FFuzz performs better than Atheris primarily because relying solely on CPU-side crashes can miss certain vulnerabilities, whereas FFuzz's enhanced oracle allows it to capture these otherwise overlooked issues. Although FFuzz-a also adopts the enhanced oracle, it applies the high-overhead detection to all of the test cases, failing to balance effectiveness and efficiency; this leads to unacceptable throughput degradation under limited computational resources. In addition, while FFuzz-c and FFuzz-n attempt to distinguish test cases, their use of edge execution counts or newly triggered edges only provides a coarse approximation of execution paths. Consequently, they may inaccurately differentiate inputs that are critical for exposing potential vulnerabilities.

5.4. Effectiveness of execution path

During the fuzzing process, FFuzz employs an enhanced oracle to significantly improve its ability to detect vulnerabilities. To avoid the unacceptable overhead associated with applying expensive checks to every test case, FFuzz differentiates inputs based on function-level execution paths and applies high-overhead analyses only to those test

Table 3
Vulnerabilities information discovered by FFuzz and baselines.

Tools		PyTorch	TensorFlow	Avg.
Atheris	Vuls	13.4	5.7	9.6
	S-vuls	0	0	0
	G-vuls	0	0	0
FreeFuzz	Vuls	6.3	3.4	4.9
	S-vuls	0	0	0
	G-vuls	0	0	0
FFuzz-a	Vuls	3.5	1.8	5.7
	S-vuls	2	1.3	1.7
	G-vuls	1.2	0.4	0.8
FFuzz-c	Vuls	19.6	9.5	14.6
	S-vuls	9.6	5.8	7.7
	G-vuls	5.8	3.2	4.5
FFuzz-n	Vuls	18.6	9.4	14
	S-vuls	6.1	4.4	5.3
	G-vuls	2.6	1.9	2.3
FFuzz	Vuls	27.3	16.2	21.8
	S-vuls	15.6	11.4	13.5
	G-vuls	7.8	5.6	6.7

Table 4
Execution information accross FFuzz and baselines.

Tools		PyTorch	TensorFlow	Avg.
Atheris	T-runs	105,840	160,560	133,200
	S-runs	0	0	0
	Ratio	0%	0%	0%
FreeFuzz	T-runs	50,005	58,253	54,129
	S-runs	0	0	0
	Ratio	0%	0%	0%
FFuzz-a	T-runs	1255	2059	1657
	S-runs	1255	2059	1657
	Ratio	100%	100%	100%
FFuzz-c	T-runs	11,801	13,805	12,803
	S-runs	1137	1174	1155
	Ratio	9.6%	8.5%	9.1%
FFuzz-n	T-runs	92,107	148,852	120,479
	S-runs	149	207	178
	Ratio	0.2%	0.1%	0.15%
FFuzz	T-runs	81,537	132,088	106,812
	S-runs	276	340	308
	Ratio	0.3%	0.25%	0.28%

cases that are more likely to trigger potential vulnerabilities. The enhanced oracle effectively uncovers vulnerabilities that would otherwise be missed by conventional crash-based detection. Table 3 summarizes the vulnerabilities discovered by FFuzz and the baseline methods. Here, S-vuls denotes vulnerabilities detected by ASan or CSan, and G-vuls represents those detected only on the GPU backend by CSan.

Compared with Atheris, which relies solely on CPU-side crashes as its oracle, FFuzz benefits substantially from enabling ASan and CSan, allowing it to reveal many previously missed issues. Among the 21.8 vulnerabilities detected on average by FFuzz, 13.5 are found only with sanitizer support, and 6.7 of these are revealed exclusively through GPU-side execution enabled by CSan. Overall, vulnerabilities exposed through the enhanced oracle account for 61.9% of FFuzz's total findings, while the proportions for FFuzz-c and FFuzz-n are 52.7% and 37.8%, respectively.

It is crucial to avoid applying expensive analysis to all test cases. By allocating resources preferentially to a subset of inputs that are more likely to expose vulnerabilities, a better balance between effectiveness and efficiency can be achieved. Table 4 presents the execution statistics for the various methods. T-runs denotes the total number of executed test cases, S-runs denotes the number of test cases executed with sanitizers enabled, and Ratio represents the proportion. Atheris achieves the highest execution throughput because it uses only crash-based detection, but its inherent false negatives result in limited vulnerability discovery capabilities. In contrast, FFuzz-a enables costly checks on all test cases, causing a sharp drop in execution throughput under constrained resources and severely reducing fuzzing efficiency.

FFuzz-c and FFuzz-n approximate execution paths using edge execution counts or newly triggered edges, which improves detection to some extent. However, to more accurately identify high-value test cases, FFuzz adopts function-level execution paths for classification. The results show that FFuzz achieves the best balance between detection capability and throughput; even though only 0.28% of its test cases are assigned to the high-overhead analysis subset, FFuzz is still able to detect vulnerabilities effectively.

Overall, by combining an enhanced oracle with function-level execution path guided test case classification, FFuzz achieves an optimal trade-off between vulnerability detection capability and fuzzing efficiency.

5.5. Effectiveness of energy allocation

To evaluate the impact of the energy distribution strategy on the fuzzing performance of FFuzz, we compare in Table 5 the version without this strategy (FFuzz-e) against the default FFuzz implementation. The experimental results show that, on average, FFuzz-e achieves a coverage of 12,863, which is approximately 10.1% lower than the 14,310 achieved by the default implementation. In terms of vulnerability discovery, FFuzz-e detects an average of 18.2 vulnerabilities, compared with 21.8 for FFuzz, representing a 16.5% reduction. These results indicate that the energy distribution strategy plays a crucial role in improving both coverage and vulnerability discovery capability.

FFuzz's energy distribution mechanism optimizes the allocation of testing resources by comprehensively evaluating the potential of each seed's mutations. Specifically, the mechanism allocates energy according to a seed's potential to generate mutated test cases that reach previously unexplored code regions, thereby promoting broader code exploration and improving coverage. In addition, it considers the potential of a seed's mutations to trigger execution paths that may reveal abnormal behaviors and potential vulnerabilities. By balancing exploration breadth and vulnerability-triggering depth, FFuzz significantly enhances the overall effectiveness of the fuzzing process.

5.6. Overhead

The runtime overhead reported in this work measures the additional cost introduced by FFuzz's function-level execution path instrumentation and the corresponding path detection logic. It does not include the runtime overhead of sanitizers, as sanitizer overhead is orthogonal to our framework's design.

The overhead of function-level execution path instrumentation is measured by executing the same set of test inputs retained during fuzzing on the target program with and without the additional instrumentation, and comparing the execution time. The overhead of determining whether a test case triggers a new function-level execution path is measured by analyzing the extra time spent on this logic during fuzzing.

Table 6 summarizes the overall performance overhead of FFuzz. Experimental results show that, on average, the combined overhead introduced by instrumentation and path detection results in approximately a 1.4% reduction in execution speed.

In contrast, the approach that approximates execution paths using edge execution counts requires no additional instrumentation, and its

Table 5
Effectiveness of energy allocation.

Target	FFuzz		FFuzz-e	
	Cov	Vuls	Cov	Vuls
PyTorch	10,583	27.3	9313	22.5
TensorFlow	18,038	16.2	16,414	13.8
Avg.	14,310	21.8	12,863	18.2

Table 6
Performance overhead of FFuzz.

Target	FFuzz-c	FFuzz-n	FFuzz		
			Instm	Detect	Sum
PyTorch	0.3%	0%	0.6%	0.9%	1.5%
TensorFlow	0.4%	0%	0.5%	0.8%	1.3%
Avg.	0.35%	0%		1.4%	

detection logic is relatively simple, resulting in a total overhead of only 0.35%. Moreover, in traditional coverage-guided fuzzing, coverage information is inherently obtained during execution without requiring additional detection steps. Therefore, approximating execution paths by checking whether a new edge is triggered introduces virtually no extra overhead.

Although FFuzz introduces about 1.4% overhead due to function-level path detection, this cost is acceptable given the significant improvement it brings to vulnerability detection capability. In other words, FFuzz trades a modest performance overhead for more effective execution-path feedback and stronger bug-triggering behavior, ultimately achieving a favorable balance between performance and effectiveness.

6. Discussion

In the current landscape of fuzzing research, various types of coverage feedback metrics are widely employed to guide the testing process and identify test cases with higher exploration value. For example, Angora (Chen and Chen, 2018) enhances edge coverage feedback by incorporating context-sensitive call stacks, thereby improving its ability to capture complex control-flow behaviors. Ijon (Aschermann et al., 2020) leverages expert-defined annotations to assist in exploring the program state space. DDFuzz (Mantovani et al., 2022) and DataFlow (Herrera et al., 2023) further investigate data-flow coverage to capture execution behaviors that traditional control-flow coverage may fail to observe. Fundamentally, these coverage metrics are used to estimate the value of test cases, determining which ones should be retained in the corpus to continuously drive the fuzzing process forward.

In contrast, FFuzz adopts a different design philosophy. Rather than focusing on corpus retention, its core objective is to distinguish test cases that are more likely to trigger vulnerabilities and apply more expensive detection mechanisms specifically to these high-value candidates. Although this design introduces slightly higher runtime overhead compared to traditional approaches, the significant improvement in vulnerability discovery demonstrates the effectiveness of the strategy.

In addition, the design of FFuzz is compatible with many existing fuzzing techniques. Approaches such as seed scheduling, mutation optimization, and learning-based input generation can be naturally integrated with FFuzz, forming a more synergistic fuzzing framework. Combining FFuzz with these approaches could provide additional benefits in terms of performance and vulnerability detection capability.

In this work, we evaluate FFuzz on PyTorch and TensorFlow, two representative and widely adopted frameworks for heterogeneous platforms. Although the evaluation is limited to these two frameworks, the design of FFuzz is largely framework-agnostic. FFuzz relies on function-level execution path analysis, sanitizer-based enhanced oracles, and adaptive energy allocation, all of which are applicable to a broad class of heterogeneous software.

For other frameworks that provide unified APIs with CPU and GPU backends, we expect FFuzz to exhibit comparable improvements in vulnerability detection, particularly in scenarios where traditional crash-based oracles suffer from limited feedback on GPU devices. However, the effectiveness of FFuzz may depend on the degree of consistency between CPU and GPU execution logic and the extent to which CPU-side execution paths can meaningfully indicate GPU-side behaviors. Evaluating FFuzz on a wider range of heterogeneous frameworks remains an important direction for future work.

7. Limitations

Despite its effectiveness, FFuzz has several limitations. First, to balance detection capability and testing efficiency, FFuzz relies on function-level execution paths to estimate the likelihood that a test case may trigger vulnerabilities. While this abstraction significantly reduces analysis overhead and improves scalability, it does not capture combinations of execution paths across functions as well as finer-grained behaviors within functions. As a result, FFuzz may miss certain vulnerability-triggering behaviors. Moreover, FFuzz may fail to capture errors that do not manifest through control-flow changes but instead arise purely from subtle data-flow interactions. Such data-dependent faults can lead to incorrect program states without altering the observed execution path.

In addition, FFuzz primarily targets common security issues such as crashes and memory errors, and it is less capable of detecting functional errors or performance issues such as execution slowdowns. Although differential testing can detect functional anomalies and may also expose vulnerabilities missed by sanitizers, it often produces false positives that require substantial manual analysis. Therefore, FFuzz does not adopt differential detection as its default strategy, but we view it as a complementary technique worth further investigation.

Finally, although FFuzz employs sanitizer-based enhanced oracles to mitigate the false negatives of traditional crash-based detection, sanitizers can only reduce, rather than eliminate, missed detections. Consequently, FFuzz may still fail to identify certain violations or vulnerability-triggering behaviors that are not covered by the employed sanitizers.

8. Future work

Our work primarily focuses on optimizing seed evaluation and energy allocation by applying fine-grained oracles to test cases that are more likely to expose vulnerabilities. An important direction for future research is to explore more effective vulnerability indicators and test oracles that can capture a broader range of program anomalies, including functional inconsistencies and performance-related issues, while maintaining acceptable performance overhead. In addition, we plan to extend FFuzz to a wider range of heterogeneous frameworks beyond PyTorch and TensorFlow to further investigate its generality and practical applicability.

9. Related work

Test oracle. A test oracle determines whether the behavior of the PUT satisfies expected correctness conditions. The most common oracle used in fuzzing is crash detection, which flags memory safety errors. Beyond simple crash-based oracles, sanitizer-assisted oracles have been introduced to enhance vulnerability detection capability. Dynamic instrumentation tools such as ASan and UBSan can detect subtle memory errors at runtime, significantly increasing the vulnerability detection capability during fuzzing.

To detect inconsistency-related defects, FreeFuzz (Wei et al., 2022) performs differential testing by executing semantically equivalent code on CPU and GPU backends and comparing their outputs. Metamorphic

testing has also been explored as a powerful oracle. By defining metamorphic relations, new test cases are generated, and correctness is verified by checking whether those relations hold. Chen et al. (2018).

Beyond functional correctness, several fuzzers integrate performance related oracles. PerfFuzz (Lemieux et al., 2018) targets pathological inputs that trigger worst-case performance behaviors. SlowFuzz (Petsios et al., 2017) automatically generates inputs that maximize resource consumption, such as execution time or memory usage, allowing it to discover performance bottlenecks or algorithmic complexity vulnerabilities.

FFuzz focuses on detecting cross-platform vulnerabilities, including those manifesting in GPU execution backends by integrating both crash-based and sanitizer-assisted oracles. Rather than solely enhancing vulnerability detection, FFuzz emphasizes a balance between detection capability and runtime overhead, enabling efficient discovery of both traditional memory errors and GPU-specific execution anomalies in heterogeneous computing environments.

Energy allocation. During the fuzzing process, most coverage-guided fuzzers assign energy to each seed, where energy represents the number of mutations scheduled for the selected seed in the current fuzzing round. To address different fuzzing scenarios, a variety of optimization strategies have been proposed to improve either code coverage or bug-finding capability. For example, AFLFast (Böhme et al., 2016) allocates more energy to less frequently executed paths to accelerate the exploration of low-probability states and thereby increase coverage. VUzzer (Rawat et al., 2017) models the fuzzing process as a Markov chain to compute optimal energy distribution, allowing seeds on promising execution paths to receive more mutation opportunities.

Beyond control-flow-based approaches, many methods explore adaptive or learning-based energy allocation strategies. FairFuzz (Lemieux and Koushik, 2018) prioritizes seeds that increase branch hit rarity, thus biasing energy toward inputs likely to reach unexplored branches. Machine-learning-based fuzzers such as NEUZZ (She et al., 2019) leverage neural guidance or meta-heuristic algorithms to estimate the importance of seeds, indirectly influencing how energy is assigned. In directed fuzzing, AFLRun (Rong et al., 2024) distributes energy fairly across seeds associated with different target locations, maximizing the likelihood of triggering vulnerabilities across multiple targets.

Compared with these approaches, FFuzz adopts a fundamentally different energy allocation strategy. Rather than focusing solely on coverage improvement or structural rarity, FFuzz jointly considers each seed's potential contribution to both coverage expansion and vulnerability triggering. By integrating these two dimensions, FFuzz can allocate energy more holistically, enabling promising seeds to receive more mutation operations, thereby enhancing bug-finding capability.

10. Conclusion

This paper presents FFuzz, a fuzzing framework designed to enhance vulnerability detection for software executed across heterogeneous platforms. FFuzz incorporates sanitizer-based checking to mitigate false negatives on the CPU side and addresses the lack of crash feedback commonly encountered in GPU execution backends. Leveraging function-level execution paths, FFuzz identifies inputs that are more likely to trigger vulnerabilities and selectively applies high-overhead checks to a subset of test cases. Combined with an adaptive energy allocation strategy, FFuzz achieves an effective balance between detection capability and runtime overhead. We implemented a prototype of FFuzz, and our experimental results demonstrate that FFuzz improves vulnerability detection capability by 227%, while introducing only 1.4% performance overhead. In addition, FFuzz uncovered six previously unknown vulnerabilities, all of which have been confirmed by developers after responsible disclosure.

CRediT authorship contribution statement

Xu Chen: Writing – original draft, Software, Methodology, Conceptualization; **Zhe Pan:** Writing – review & editing; **Ruomin Fang:** Data curation; **Liwei Chen:** Supervision; **Gang Shi:** Funding acquisition.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This research is supported by the Youth Innovation Promotion Association CAS, China and National Natural Science Foundation of China (No. 62172407).

References

Aschermann, C., Schumilo, S., Abbasi, A., Holz, T., 2020. IJON: exploring deep state spaces via fuzzing. In: 2020 IEEE Symposium on Security and Privacy (SP), pp. 1597–1612. https://doi.org/10.1109/SP40000.2020.00117

Böhme, M., Pham, V.-T., Roychoudhury, A., 2016. Coverage-based greybox fuzzing as Markov chain. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, pp. 1032–1043. https://doi.org/10.1145/2976749.2978428

Chen, P., Chen, H., 2018. Angora: efficient fuzzing by principled search. In: 2018 IEEE Symposium on Security and Privacy (SP), pp. 711–725. https://doi.org/10.1109/SP.2018.00046

Chen, T.Y., Kuo, F.-C., Liu, H., Poon, P.-L., Towey, D., Tse, T.H., Zhou, Z.Q., 2018. Metamorphic testing: a review of challenges and opportunities. ACM Comput. Surv. 51 (1). https://doi.org/10.1145/3143561

Chen, X., Cui, N., Pan, Z., Chen, L., Shi, G., Meng, D., 2025. Critical variable state-aware directed greybox fuzzing. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE Computer Society, Los Alamitos, CA, USA, pp. 141–152. https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00219. https://doi.org/10.1109/ICSE55347.2025.00219

Corporation, N., 2023. Nvidia compute sanitizer. Accessed June 22, 2025. https://developer.nvidia.com/compute-sanitizer.

Deng, Y., Xia, C.S., Peng, H., Yang, C., Zhang, L., 2023. Large language models are zero-shot fuzzers: fuzzing deep-learning libraries via large language models. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. Association for Computing Machinery, New York, NY, USA, pp. 423–435. https://doi.org/10.1145/3597926.3598067

Fioraldi, A., Maier, D., Eißfeldt, H., Heuse, M., 2020. AFL++ : combining incremental steps of fuzzing research. In: 14th USENIX Workshop on Offensive Technologies (WOOT 20). USENIX Association. https://www.usenix.org/conference/woot20/presentation/fioraldi.

Ganesh, V., Leek, T., Rinard, M., 2009. Taint-based directed whitebox fuzzing. In: 2009 IEEE 31st International Conference on Software Engineering, pp. 474–484. https://doi.org/10.1109/ICSE.2009.5070546

Google, 2020. Google. atheris: a coverage-guided, native Python fuzzer. Accessed June 22, 2025. https://github.com/google/atheris.

Guo, Y., Zhang, Z., Yang, J., 2024. GPU memory exploitation for fun and profit. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, Philadelphia, PA, pp. 4033–4050. https://www.usenix.org/conference/usenixsecurity24/presentation/guo-yanan.

Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M.H., Brett, M., Haldane, A., del Río, J.F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., Oliphant, T.E., 2020. Array programming with NumPy. Nature 585 (7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Herrera, A., Payer, M., Hosking, A.L., 2023. DatAFLow: toward a data-flow-guided fuzzer. ACM Trans. Softw. Eng. Methodol. 32 (5). https://doi.org/10.1145/3587156

Inc, G., 2016. Syzkaller-kernel fuzzer. Accessed June 22, 2025. https://github.com/google/syzkaller.

Jiang, Z., Jiang, X., Hazimeh, A., Tang, C., Zhang, C., Payer, M., 2021. IGOR: crash deduplication through root-cause clustering. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, pp. 3318–3336. https://doi.org/10.1145/3460120.3485364

Kong, Z., Li, S., Huang, H., Su, Z., 2025. Sand: decoupling sanitization from fuzzing for low overhead. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE Computer Society, Los Alamitos, CA, USA, pp. 255–267. https://doi.org/10.1109/ICSE55347.2025.00187

Lee, B., Song, C., Kim, T., Lee, W., 2015. Type casting verification: stopping an emerging attack vector. In: 24th USENIX Security Symposium (USENIX Security 15). USENIX Association, Washington, D.C., pp. 81–96. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/lee.

Lemieux, C., Koushik, S., 2018. Fairfuzz: a targeted mutation strategy for increasing greybox fuzz testing coverage. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering. Association for Computing Machinery, New York, NY, USA, pp. 475–485. https://doi.org/10.1145/3238147.3238176

Lemieux, C., Padhye, R., Koushik, S., Song, D., 2018. PerfFuzz: automatically generating pathological inputs. In: Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis. Association for Computing Machinery, New York, NY, USA, p. 254-265. https://doi.org/10.1145/3213846.3213874

Li, Z., Wu, J., Ling, X., Luo, T., Rui, Z., Wu, Y., 2025. The seeds of the future sprout from history: fuzzing for unveiling vulnerabilities in prospective deep-learning libraries. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE Computer Society, Los Alamitos, CA, USA, pp. 1616–1627. https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00132. https://doi.org/10.1109/ICSE55347.2025.00132

Mantovani, A., Fioraldi, A., Balzarotti, D., 2022. Fuzzing with data dependency information. In: 2022 IEEE 7th European Symposium on Security and Privacy (EuroSP), pp. 286–302. https://doi.org/10.1109/EuroSP53844.2022.00026

Petsios, T., Zhao, J., Keromytis, A.D., Jana, S., 2017. SlowFuzz: automated domain-independent detection of algorithmic complexity vulnerabilities. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, p. 2155-2168. https://doi.org/10.1145/3133956.3134073

Rawat, S., Jain, V., Kumar, A., Cojocar, L., Giuffrida, C., Bos, H., 2017. VUzzer: Application-aware evolutionary fuzzing. In: 2017 Network and Distributed System Security (NDSS) Symposium. Internet Society, pp. 1–14. https://doi.org/10.14722/ndss.2017.23404

Rong, H., You, W., Wang, X., Mao, T., 2024. Toward unbiased multiple-target fuzzing with path diversity. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, Philadelphia, PA, pp. 2475–2492. https://www.usenix.org/conference/usenixsecurity24/presentation/rong.

Serebryany, K., 2012. A memory error detector. Accessed June 22, 2025. https://github.com/google/sanitizers/wiki/AddressSanitizer.

Serebryany, K., 2017. Libfuzzer: A library for coverage-guided fuzztesting (within llvm). Accessed June 22, 2025. https://llvm.org/docs/LibFuzzer.{html}.

She, D., Pei, K., Epstein, D., Yang, J., Ray, B., Jana, S., 2019. Neuzz: efficient fuzzing with neural program smoothing. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 803–817. https://doi.org/10.1109/SP.2019.00052

Stone, J.E., Gohara, D., Shi, G., 2010. OpenCL: a parallel programming standard for heterogeneous computing systems. Comput. Sci. Eng. 12 (3), 66–73.

Team, G.B., 2015. Tensorflow. Accessed June 22, 2025. https://www.tensorflow.org/.

Team, P., 2017. Pytorch. Accessed June 22, 2025. http://pytorch.org/.

Wang, J., Duan, Y., Song, W., Yin, H., Song, C., 2019. Be sensitive and collaborative: analyzing impact of coverage metrics in greybox fuzzing. In: 22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019). USENIX Association, Chaoyang District, Beijing, pp. 1–15.

Wei, A., Deng, Y., Yang, C., Zhang, L., 2022. Free lunch for testing: fuzzing deep-learning libraries from open source. In: Proceedings of the 44th International Conference on Software Engineering. Association for Computing Machinery, New York, NY, USA, pp. 995–1007. https://doi.org/10.1145/3510003.3510041

Woo, M., Cha, S.K., Gottlieb, S., Brumley, D., 2013. Scheduling black-box mutational fuzzing. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security. Association for Computing Machinery, New York, NY, USA, pp. 511–522. https://doi.org/10.1145/2508859.2516736

You, W., Wang, X., Ma, S., Huang, J., Zhang, X., Wang, X., Liang, B., 2019. ProFuzzer: on-the-fly input type probing for better zero-day vulnerability discovery. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 769–786. https://doi.org/10.1109/SP.2019.00057

Zalewski, M., 2014. American fuzzy lop. https://github.com/google/AFL.

Zheng, H., Zhang, J., Huang, Y., Ren, Z., Wang, H., Cao, C., Zhang, Y., Toffalini, F., Payer, M., 2023. FISHFUZZ: catch deeper bugs by throwing larger nets. In: 32nd USENIX Security Symposium (USENIX Security 23). USENIX Association, Anaheim, CA, pp. 1343–1360.